



DTO & Business Layer

2026

Profesores:

- Dr. Freddy Paz
- Dr. José Luis Corcuera
- Mag. Eric Huiza
- Mag. David Allasi

1. La interfaz `java.sql.CallableStatement`
2. DTO
3. Business Layer
4. Definición de módulos en el proyecto SoftProg

AGENDA



La interfaz `java.sql.CallableStatement`



La interfaz `java.sql.PreparedStatement`

- Es una interfaz que extiende la funcionalidad de `PreparedStatement`.
- Se utiliza específicamente para llamar y ejecutar procedimientos almacenados (stored procedures) directamente en la base de datos.
- A diferencia de otras sentencias, está diseñado para manejar tanto parámetros de entrada como de salida, permitiendo enviar datos al procedimiento y recuperar los resultados calculados por este.
- Al ser una extensión de sentencias previas, también puede generar objetos `ResultSet` al ejecutarse y soporta la agrupación de operaciones para actualizaciones por lotes (Statement y `PreparedStatement` soportan también actualización por lotes).



La interfaz `java.sql.PreparedStatement`

- Se define el siguiente procedimiento almacenado en MySQL el cual recibe dos parámetros:
 - `_id_area` el cual es un parámetro OUT
 - `_nombre` el cual es un parámetro IN

```
DELIMITER $  
CREATE PROCEDURE INSERTAR_AREA(  
    OUT _id_area INT,  
    IN _nombre VARCHAR(75)  
)BEGIN  
    INSERT INTO area(nombre,activa) VALUES(_nombre,1);  
    SET _id_area = @@last_insert_id;  
END$
```



La interfaz `java.sql.PreparedStatement`

```
import java.sql.CallableStatement;
import java.sql.SQLException;
import java.sql.Types;

// we've already got a connection

try (CallableStatement cs = connection.prepareCall("{CALL INSERTAR_AREA(?, ?)}")) {
    // Registrar el parámetro de salida (índice 1)
    cs.registerOutParameter(1, Types.INTEGER);

    // Establecer el parámetro de entrada (índice 2)
    cs.setString(2, "Sistemas");

    // Ejecutar
    cs.execute();

    // Recuperar el ID generado
    int idGenerado = cs.getInt(1);
    System.out.println("ID del área insertada: " + idGenerado);
}

cs.close();
connection.close();
```



La interfaz `java.sql.PreparedStatement`

- Se define el siguiente procedimiento almacenado en MySQL el cual recibe un parametro:
 - `_id_area` el cual es un parámetro IN

```
DELIMITER $  
CREATE PROCEDURE OBTENER_AREA_X_ID(  
    IN _id_area INT  
)BEGIN  
    SELECT id_area, nombre, activa FROM area WHERE id_area =  
    _id_area;  
END$
```



La interfaz `java.sql.PreparedStatement`

```
import java.sql.ResultSet;
import java.sql.CallableStatement;
import java.sql.SQLException;
import java.sql.Types;

// we've already got a connection
try (CallableStatement cs = connection.prepareCall("{CALL OBTENER_AREA_X_ID(?)}")) {

    // Establecer el parámetro de entrada (IN)
    cs.setInt(1, idBusqueda);

    // Ejecutar y obtener el conjunto de resultados (ResultSet)
    try (ResultSet rs = cs.executeQuery()) {

        if (rs.next()) { // Si existe al menos una fila
            // Extraer los datos por nombre de columna o índice
            int id = rs.getInt("id_area");
            String nombre = rs.getString("nombre");
            boolean activa = rs.getBoolean("activa");
            System.out.println("Área Encontrada> ID: " + id + ", Nombre: " + nombre + ", Activa: " + activa);
        } else {
            System.out.println("No se encontró ninguna área con el ID: " + idBusqueda);
        }
    }
}
```



Data Transfer Object (DTO)



Data Transfer Object (DTO)

- Dado que estamos trabajando con OOP un registro de una tabla corresponderá a un objeto de la clase que representa esa tabla, por ejemplo si tengo mi tabla empleado y el registro de ID 5 eso quiere decir que en mi aplicación tendré la clase Empleado y una instancia de ID 5 (la cuál representa el registro en mención).

¿A qué clase mapearía los resultados de la ejecución de este query en donde por cada año - mes se reporta el monto total de ventas y cantidad de productos?



Data Transfer Object (DTO)

Query:

```
select
    year(o.fecha_hora) as _year,
    month(o.fecha_hora) as _month,
    sum(o.total) as _total,
    sum(lov.cantidad) as cantidad
from
    orden_venta o
    inner join linea_orden_venta lov on lov.fid_orden_venta =
o.id_orden_venta
group by
    year(o.fecha_hora),
    month(o.fecha_hora)
order by
    1 desc, 2 desc;
```

**Se requiere de una clase
que mapee la estructura de
la tupla resultante de este
query -> DTO**



Data Transfer Object (DTO)

- Para responder a la pregunta anterior se requiere definir un Data Transfer Object (DTO).
- El DTO es una clase que solo contiene los atributos específicos que necesitas mover de un lugar a otro.
- Para el query anterior tendría que definir el DTO:

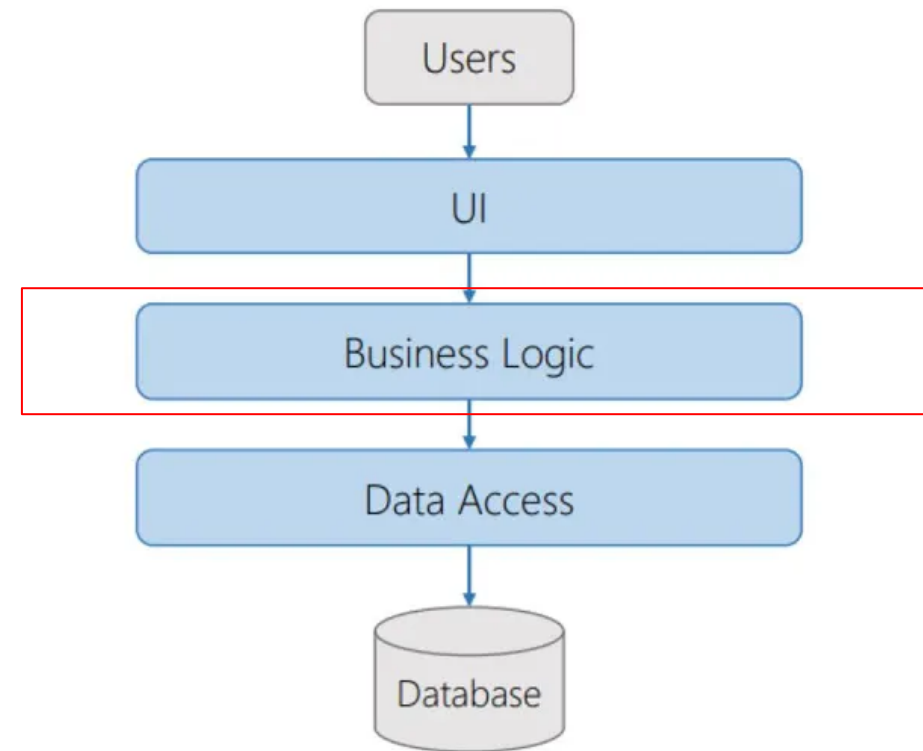
```
class SalesReportRecordDTO {  
    private Integer year;  
    private Integer month;  
    private Double totalAmount;  
    private Integer quantity;  
}
```



Business Layer

Business Layer

- Esta capa es el "corazón" del software, donde reside la lógica que define cómo funciona el negocio, independientemente de cómo se guarden los datos (Base de Datos) o cómo se muestran (UI).



Business Layer

- Generación de ordenes de ventas en SoftProg:
 - El empleado genera una orden de venta con la información de los productos que el cliente desea adquirir y, por cada producto, se especifica la cantidad de este.
 - El sistema recibe la orden de venta y procede de la siguiente forma:
 - Valida que el empleado este activo
 - Valida que el cliente exista
 - Por cada producto se válida si hay stock disponible. En caso de no tener stock se emite el mensaje de error respectivo.
 - Si hay stock en todos los producto se procede a crear la orden de venta y sus líneas de orden de venta
 - Por cada producto se actualiza el stock de estos
 - Como paso final, se emite el número (o ID) de la orden de venta generada.

Business Layer

- Generación de ordenes de ventas en SoftProg:

```
interface OrdenVentaBL {  
  
    void registrar(OrdenVenta ordenVenta);  
    // otros metodos aqui  
  
}
```


Business Layer

- Generación de ordenes de ventas en SoftProg:

```
class OrdenVentaBLImpl implements OrdenVentaBL {  
  
    public void registrar(OrdenVenta ordenVenta) {  
        // 1. La operacion de registro de orden de venta debe ser indivisible  
        // 2. Valida que el empleado exista y este activo  
        // 3. Valida que el cliente exista  
        // 4. Se valida el stock. Si no hay stock suficiente, se debería lanzar una Exception  
        // 5. Se procede a registrar la orden de venta y sus lineas  
        // 6. Se procede a actualizar el stock de los productos  
        // 7. Confirmo la t.....  
    }  
}
```

¿Se necesita de los objetos de la capa DAO?
¿Cómo garantizamos el punto (1) de los comentarios?



Business Layer

- Generación de ordenes de ventas en SoftProg:

Respondiendo a las preguntas:

¿Se necesita de los objetos de la capa DAO?

Si, dado que necesitamos validar el stock de los productos necesitamos cargar la información de estos además de validar que el empleado este activo y que el cliente exista.

¿Cómo garantizamos el punto (1) de los comentarios?

Cuando se menciona indivisible se refiere a que toda esa lógica se ejecute el todo o nada y esto lo garantizamos haciendo uso de transacciones. Entonces la idea es **usar la misma Connection a lo largo de la ejecución de la operación de negocio.**

Definición de módulos en el proyecto SoftPro

Definición de módulos en el proyecto SoftPro

- A la fecha hemos definido los siguientes módulos:
 - **softprog-model.**
 - **softprog-dao.**
 - **softprog-db-manager.**
- La lógica de negocio sera implementada en el módulo:
 - **softprog-business.**

Definición de módulos en el proyecto SoftPro

Demo

¡Gracias!

